MediLink LK – AI-Enabled Smart Healthcare Appointment & Telemedicine Platform

SE3020 – Distributed Systems (Assignment 1) • Report • Date: 2026-04-17

Repository scope: Microservices-based healthcare platform demonstrating service discovery, API gateway with JWT/RBAC, per-service databases, async messaging, and end-to-end workflows (appointments → payments → telemedicine → notifications → prescriptions).

Deliverables: This report is designed to be exported as `report.pdf`. It contains no screenshots; code excerpts are included as plain text.

Table of Contents

1. Introduction

MediLink LK is a distributed, microservices-based healthcare platform concept that supports doctor onboarding/verification, appointment booking, online payments, telemedicine session provisioning, consultation completion, patient notifications, and digital prescriptions.

Goals (assignment-aligned):

- Demonstrate microservices separation and independent deployment
- Use API Gateway + Service Discovery for routing
- Implement JWT authentication and role-based access control
- Use asynchronous messaging (RabbitMQ) for event-driven workflows
- Maintain per-service data ownership (PostgreSQL per service)
- Provide a repeatable end-to-end proof run (smoke test)

2. Architecture (High-level)

Key components:

- **Frontend** (React + Vite, served via Nginx)
- **API Gateway** (Spring Cloud Gateway) for routing + JWT validation + RBAC
- **Service Discovery** (Eureka) for dynamic registration
- **Domain services**: auth, patient, doctor, appointment, payment, telemedicine, notification, prescription, symptom checker

- **RabbitMQ** for asynchronous events
- **PostgreSQL** per service for data ownership

2.1 Architectural Diagram (text)

```
Browser
  │
  ▼
Frontend (5173)
  │  HTTP (/api)
  ▼
API Gateway (8090)  ──────────────► Eureka (8761)
  │   routes (lb://...)             ▲
  ├──────────► auth-service (8081)    │
  ├──────────► patient-service (8086) │
  ├──────────► doctor-service (8084)  │
  ├──────────► appointment-service (8082) ── publish ──► RabbitMQ (5672)
  ├──────────► payment-service (8085)       publish ──► RabbitMQ (5672)
  ├──────────► telemedicine-service (8087)  publish ──► RabbitMQ (5672)
  ├──────────► prescription-service (8088)
  └──────────► symptom-checker-service (8089) ──► (optional) ai-symptom-service (8010)

RabbitMQ consumers:
  notification-service (8083)  ◄── consumes events, stores notifications

Databases (PostgreSQL): authdb, patientdb, doctordb, appointmentdb, paymentdb, telemedicin
                        prescriptiondb, notificationdb, symptomdb
```

3. Service Interfaces (API contracts)

All endpoints are accessed through the API Gateway in normal operation. Identity is injected by the gateway into `X-User-Id` and `X-User-Role` headers.

3.1 Auth Service

- `POST /api/auth/register` – Register and receive JWT
- `POST /api/auth/login` – Login and receive JWT
- `GET /api/admin/users` – Admin list/search users
- `DELETE /api/admin/users/{id}` – Admin delete a non-admin user

3.2 Doctor Service

- `POST /api/doctors/me/profile` – Upsert doctor profile
- `POST /api/doctors/me/profile-photo` – Upload doctor profile photo
- `GET /api/doctors/me/profile` – Get own profile
- `GET /api/doctors` – Search verified doctors (optional specialization filter)
- `GET /api/doctors/{doctorId}/availability` – Get availability blocks

- `GET /api/doctors/{doctorId}/availability/slots` – Generate slots
- `GET /api/doctors/{doctorId}/availability/validate` – Validate a proposed slot time
- `GET /api/admin/doctors/pending` – Admin list pending verifications
- `GET /api/admin/doctors/recent-decisions` – Admin recent approve/reject decisions
- `POST /api/admin/doctors/{doctorId}/approve` – Admin approve doctor
- `POST /api/admin/doctors/{doctorId}/reject` – Admin reject doctor

3.3 Patient Service

- `GET /api/patients/me/profile` – Get own patient profile
- `POST /api/patients/me/profile` – Upsert patient profile
- `POST /api/patients/me/profile-photo` – Upload patient profile photo
- `POST /api/patients/me/reports` – Upload medical report
- `GET /api/patients/me/reports` – List reports
- `GET /api/patients/me/reports/{id}/download` – Download report
- `DELETE /api/patients/me/reports/{id}` – Delete report
- `GET /api/admin/patients` – Admin list patient profiles
- `GET /api/admin/patients/{userId}/profile` – Admin get a patient profile
- `GET /api/admin/patients/{userId}/reports` – Admin list patient reports
- `GET /api/admin/patients/{userId}/reports/{id}/download` – Admin download patient report

3.4 Appointment Service

- `POST /api/appointments` – Create appointment
- `GET /api/appointments` – List patient appointments
- `GET /api/appointments/doctor/me` – List doctor appointments
- `PUT /api/appointments/doctor/me/{id}/approval` – Doctor approve/reject an appointment
- `GET /api/appointments/available-slots` – Available slots for a doctor
- `PUT /api/appointments/{id}/reschedule` – Reschedule appointment
- `DELETE /api/appointments/{id}` – Cancel appointment
- `GET /api/appointments/stream` – Patient SSE stream
- `GET /api/appointments/doctor/me/stream` – Doctor SSE stream
- `GET /api/appointments/admin/stream` – Admin SSE stream
- `GET /api/admin/appointments` – Admin list appointments
- `DELETE /api/admin/appointments/{id}` – Admin cancel appointment

3.5 Payment Service

- `POST /api/payments/intents/payhere` – Create PayHere intent
- `POST /api/payments/intents/stripe` – Create Stripe checkout intent
- `POST /api/payments/stripe/confirm` – Confirm Stripe session
- `POST /api/payments/callback/payhere` – PayHere server-to-server notify
- `POST /api/payments/callback/stripe` – Stripe webhook
- `GET /api/admin/payments` – Admin list payments
- `POST /api/admin/payments/{paymentId}/review` – Admin review payment

- `POST /api/admin/payments/{paymentId}/dispute` – Admin flag dispute
- `POST /api/admin/payments/{paymentId}/refund` – Admin refund

3.6 Telemedicine Service

- `GET /api/telemedicine/sessions/appointment/{appointmentId}` – Get session info (join URL)
- `POST /api/telemedicine/sessions/appointment/{appointmentId}/complete` – Mark consultation completed

3.7 Notification Service

- `GET /api/notifications` – List current user notifications
- `GET /api/admin/notifications` – Admin list notifications

3.8 Prescription Service

- `POST /api/prescriptions/doctor/me` – Doctor issue prescription
- `GET /api/prescriptions/doctor/me` – Doctor list issued prescriptions
- `GET /api/prescriptions/patient/me` – Patient list prescriptions
- `GET /api/prescriptions/{id}` – Get prescription (user-scoped)
- `GET /api/admin/prescriptions` – Admin list prescriptions

3.9 Symptom Checker Service (optional enhancement)

- `POST /api/symptoms/check` – Create assessment and receive recommendations
- `GET /api/symptoms/history` – List user assessment history
- `GET /api/symptoms/{id}` – Get a specific assessment
- `GET /api/admin/symptoms` – Admin view recent assessments

4. Workflows

4.1 Doctor onboarding + admin verification

1. Doctor registers/logins via `/api/auth/*`.
2. Doctor creates profile via `POST /api/doctors/me/profile`.
3. Admin approves via `POST /api/admin/doctors/{doctorId}/approve`.

4.2 Appointment booking → payment → confirmation

1. Patient books appointment via `POST /api/appointments` (initially pending payment).
2. Doctor approves via `PUT /api/appointments/doctor/me/{id}/approval`.
3. Patient creates payment intent via `POST /api/payments/intents/*`.
4. Payment callback/webhook marks payment completed and publishes an event.
5. Appointment transitions to confirmed asynchronously (event-driven).

4.3 Telemedicine session → consultation completion → notifications

1. After appointment confirmation, telemedicine session is provisioned and can be fetched via `GET /api/telemedicine/sessions/appointment/{id}`.

2. Doctor marks completion via `POST /api/telemedicine/sessions/appointment/{id}/complete`.
3. Both patient and doctor receive `consultation.completed` notifications (verified in smoke test).

4.4 Real-time appointment status updates (SSE)

Appointment status changes are broadcast using Server-Sent Events so the UI can update in real time.

5. Security & Authentication

- JWT is issued by auth-service and validated at the API gateway.
- The gateway enforces RBAC (role checks where configured) and injects identity headers (`X-User-Id`, `X-User-Role`, optional `X-User-Email`).
- Downstream services authorize requests using these headers (least-privilege per endpoint).

6. Deployment

6.1 Docker Compose

```
docker compose up --build
```

Key URLs:

- Frontend: `http://localhost:5173`
- Gateway: `http://localhost:8090`
- Eureka: `http://localhost:8761`
- RabbitMQ UI: `http://localhost:15672` (guest/guest)

6.2 Kubernetes

Manifests are under `k8s/` and include deployments/services/config/secrets/ingress plus readiness/liveness probes.

6.3 Automated proof run (smoke test)

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\smoke-test.ps1
```

The smoke test executes the complete workflow through the gateway and validates key outcomes (confirmation, session provisioning, completion notifications, admin review actions).

7. Individual Contributions

Fill this section with member details and responsibilities.

- Member 1: <name> – <areas>
- Member 2: <name> – <areas>
- Member 3: <name> – <areas>
- Member 4: <name> – <areas>

8. Appendix – Code Written (NO screenshots)

8.1 API Gateway JWT + RBAC header propagation (excerpt)

File: `api-gateway/src/main/java/lk/medilink/gateway/security/JwtAuthGatewayFilter.java`

```
package lk.medilink.gateway.security;

import io.jsonwebtoken.Claims;
import lk.medilink.gateway.security.JwtService.JwtValidationResult;
import org.springframework.cloud.gateway.filter.GatewayFilter;
import org.springframework.cloud.gateway.filter.factory.AbstractGatewayFilterFactory;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

import java.util.List;

@Component
public class JwtAuthGatewayFilter extends AbstractGatewayFilterFactory<JwtAuthGatewayFilte
        private final JwtService jwtService;

        public JwtAuthGatewayFilter(JwtService jwtService) {
                super(Config.class);
                this.jwtService = jwtService;
        }

        @Override
        public GatewayFilter apply(Config config) {
                return (exchange, chain) -> {
                        String authHeader = exchange.getRequest().getHeaders().getFirst(
                        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
                                return unauthorized(exchange, "Missing Bearer token");
                        }

                        JwtValidationResult result = jwtService.validate(authHeader.subs
                        if (!result.valid()) {
                                return unauthorized(exchange, "Invalid token");
                        }

                        Claims claims = result.claims();
                        List<String> roles = jwtService.extractRoles(claims);
                        if (config.requiredRoles != null && !config.requiredRoles.isEmpt
                                boolean ok = roles.stream().anyMatch(r -> config.requir
                                if (!ok) {
                                        return forbidden(exchange, "Insufficient role"
                                }
                        }
```

```
                        ServerWebExchange mutated = exchange.mutate()
                                        .request(r -> r.headers(h -> {
                                                h.add("X-User-Id", String.valueOf(cla
                                                h.add("X-User-Role", String.join(",",
                                                Object email = claims.get("email");
                                                if (email != null) {
                                                        h.add("X-User-Email", String
                                                }
                                        }))
                                        .build();

                        return chain.filter(mutated);
                };
        }

        // ...
}
```

8.2 Real-time appointment updates hub (excerpt)

File: `appointment-service/src/main/java/lk/medilink/appointment/realtime/AppointmentSseHub.java`

```
package lk.medilink.appointment.realtime;

import lk.medilink.appointment.domain.Appointment;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Component;
import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;

import java.io.IOException;
import java.time.Instant;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

@Component
public class AppointmentSseHub {
        private static final long EMITTER_TIMEOUT_MS = 30L * 60L * 1000L;

        private final Map<Long, Set<SseEmitter>> patientEmitters = new ConcurrentHashMap<
        private final Map<Long, Set<SseEmitter>> doctorEmitters = new ConcurrentHashMap<>
        private final Set<SseEmitter> adminEmitters = ConcurrentHashMap.newKeySet();

        public SseEmitter registerPatient(Long patientUserId) { /* ... */ }
        public SseEmitter registerDoctor(Long doctorId) { /* ... */ }
```

```
        public SseEmitter registerAdmin() { /* ... */ }

        public void publish(Appointment appt) {
                if (appt == null || appt.getId() == null) {
                        return;
                }
                // Publishes appointment events to patient + doctor + admin streams
                publishToSet(patientEmitters.get(appt.getPatientId()), appt);
                publishToSet(doctorEmitters.get(appt.getDoctorId()), appt);
                publishToSet(adminEmitters, appt);
        }

        private void publishToSet(Set<SseEmitter> emitters, Appointment appt) {
                if (emitters == null || emitters.isEmpty()) {
                        return;
                }
                String eventId = appt.getId() + "-" + Instant.now().toEpochMilli();
                SseEmitter.SseEventBuilder event = SseEmitter.event()
                                .id(eventId)
                                .name("appointment")
                                .data(appt, MediaType.APPLICATION_JSON);

                for (SseEmitter emitter : emitters) {
                        try {
                                emitter.send(event);
                        } catch (IOException ex) {
                                emitter.complete();
                        }
                }
        }
}
```

8.3 Telemedicine schema safety fix (excerpt)

File: `telemedicine-service/src/main/java/lk/medilink/telemedicine/config/TelemedicineSchemaFixes.java`

```
package lk.medilink.telemedicine.config;

import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;

@Configuration
public class TelemedicineSchemaFixes {
        @Bean
        ApplicationRunner telemedicineSchemaFixRunner(JdbcTemplate jdbc) {
```

```
                return args -> {
                        jdbc.execute("ALTER TABLE telemedicine_sessions DROP CONSTRAINT
                        jdbc.execute("ALTER TABLE telemedicine_sessions " +
                                        "ADD CONSTRAINT telemedicine_sessions_status_c
                                        "CHECK (status in ('ACTIVE','COMPLETED','CANCE
                };
        }
}
```

8.4 Smoke test verification for consultation completion (excerpt)

File: `scripts/smoke-test.ps1`

```
# Completing consultation (doctor) and verifying notifications
$completeResp = Invoke-RestMethod -Method Post -Uri "$gateway/api/telemedicine/sessions/ap

# Poll until session status becomes COMPLETED
$completedSession = Invoke-RestMethod -Method Get -Uri "$gateway/api/telemedicine/sessions

# Verify consultation.completed notifications (patient + doctor)
$pNotifs = Invoke-RestMethod -Method Get -Uri "$gateway/api/notifications?limit=50" -Heade
$dNotifs = Invoke-RestMethod -Method Get -Uri "$gateway/api/notifications?limit=50" -Heade
```

End of report.